High-Performance P2P Small Group Video Sharing

Group 14

Version 1.0 Baseline

Personnel

Austin Bray - Backend & Infrastructure Developer
braya3@mcmaster.ca

Carter Broom - Full-stack Developer
broomc@mcmaster.ca

Aurko Datta - Full-stack Developer
dattaa18@mcmaster.ca

Dhiren Chander - Backend Developer & Project Manager
chanderd@mcmaster.ca

Oliver Cameron - Backend Developer
cameroo@mcmaster.ca

Leonid Ginzburg - Frontend Developer
ginzburl@mcmaster.ca

Definitions, Acronyms & Abbreviations

SGVS: An acronym for Small Group Video Sharing. Refers to the project as a whole.
Discord: A social platform that allows communication through voice calls, video calls, and messaging.

Contributions

Team Member	Sections Authored	Sections Reviewed	Percentage of Work
Austin Bray	Sections 7, 11, and 15	Sections 9 and 17	16.67%
Carter Broom	Section 1-5, 15, and 16	Section 10 and 12	16.67%
Aurko Datta	Sections 9 and 17	Section 6 and 14	16.67%
Dhiren Chander	Sections 8 and 13	Sections 7 and 11	16.67%
Oliver Cameron	Sections 6 and 14	Sections 8 and 13	16.67%
Leonid Ginzburg	Sections 10 and 12	Sections 1-5, and 16	16.67%

Context and Overall Objectives

Casual online video call and community building applications are extremely popular among young people for chatting in audio calls, sharing in video calls, streaming video games, and much more. Discord is by far the most popular of these applications. Currently, available video sharing and online hangout call applications like Discord limit the bandwidth for screensharing, resulting in lower resolution and frame rates. The screen sharing function also can have substantial latency. All of these negatively impact the user experience, especially in instances where screen sharing latency is important, like in esports coaching. Additionally, all data also goes to Discord's servers, which infringes on the user's privacy.

High Performance P2P Small Group Video Sharing (also known as SGVS) solves these problems. Users will have fine grained control over bandwidth limits. The user gets to choose if they want higher resolution, or better frame rates and latency. The app analyzes the network mesh to maximize throughput based on user set limits and network limits. Since it is peer-to-peer, call data is not sent to a server, significantly reducing latency, and giving the users in the call complete privacy. This significantly improves the user screen sharing experience in comparison with the current available versions on the market.

SGVS will reshape the way that people hang out online. It will strengthen the relationships between friends since they will be able to have fun talking and watching each other play games on their terms. Esports coaching will be changed for the better. The low screen sharing latency will allow coaches to provide feedback faster, making a difference in cases where every second counts. Users can feel comfortable knowing that their information is secure and private since no server is used.

---

6. Stakeholders & User Personas

Primary Clients

The primary target audience for SGVS is small groups of friends or colleagues who wish to remotely share video, presentations, gameplay, and other in a higher quality than commercially available server-based screen-sharing applications currently allow. Since the connection is peer-to-peer, the number of participants before a drop in performance is limited by the upload speed of the person sharing their screen, so it is not intended to replace more enterprise-oriented communication services intended for large groups like Teams or Zoom. Thus, the target market will be gamers or other small, closed groups each of size 2-10, who have frequent virtual hangouts.

Secondary Stakeholders

These are the businesses or organizations outside our primary clients who could be impacted by the use of SGVS.

- **Discord, Teamspeak (primary competitors)**
  These applications are marketed towards groups of friends and gamers, but their limited bandwidth and client-server architecture leave a gap in the market for those concerned with video quality and privacy/data-collection respectively. Hence, our application would detract from Discord and Teamspeak's user-base.
- **Internet service providers**
  Because SGVS uses a peer-to-peer connection to avoid centralized servers, the sharing of high-bandwidth audio and video directly to other users can significantly impact local network congestion and data routing for ISPs.
- **Parsec, Teamviewer**
  These are remote desktop applications. They also utilize a high-bandwidth peer-to-peer connection which will force Parsec and Teamviewer to examine their own implementation should SGVS' be robust enough.
- **GPU vendors**
  The hardware encoders that SGVS will use depend on their drivers.

- **Law enforcement agencies**
  If SGVS is used for illicit activities, its peer-to-peer decentralized connection makes it difficult to track communications sent over the app. This makes law enforcement a secondary stakeholder, as they rely on obtaining logs from server-client businesses that lack end-to-end encryption (such as Discord) to catch these illegal activities.

End-user Archetypes

These are the theoretical profiles users of SGVS would fulfill during a typical session. The following archetypes are not mutually exclusive, and a user can be all at once should they have a reason to.

- **Users joining audio calls**
  Self-explanatory, and a prerequisite to the other end-user archetypes. They may be relaying broadcasts for other users if that feature is implemented.
- **Users joining to share their screen**
  These users are sharing a stream of their screen directly to the rest of the participants (see above) through a peer-to-peer connection.
  Users watching other user's screens
  These are call-participants opting to watch another user's screen. The remainder of participants likely fall into this category when one or more people are sharing their screen.

---

7. Constraints and Assumptions

This software is designed to provide the ability to share screens to almost everyone with a semi-modern laptop running a major operating system

- Hardware and network minimum requirements
  - Download speed of 3 Mbps (Calculated based on streaming 480p 30fps video with ten audio streams)
  - Upload speed of 2 Mbps (Calculated based on sharing 480p 30fps video and one audio stream)
  - CPU equivalent to the Intel N100
  - Any of the following operating systems: Windows 10, Windows 11, Debian, Arch, Fedora, or Mac OS (Operating systems beyond Windows are specified as P2 / P3
- Hardware and network optimal requirements
  - Download speed of 100 Mbps (Double the requirement to stream 4k 60fps video)
  - Upload speed of 100 Mbps (Double the requirement to share 4k 60fps video)

    - CPU equivalent to an Intel Core 5 Ultra or AMD Ryzen 5

The initial design requirements include streaming directly from one user to every other user in the call. Due to this a maximum of ten viewers per stream will be enforced in order to maintain quality for all users in the stream. Along with this, cloud infrastructure will not be used in this project as the calls / streams will be done through peer to peer connections between users.

This application will not collect or store any user data permanently on non-user devices. The following user data will be shared between users in order to execute the basic functionalities of the apps: Audio, Screen captures, IP Addresses, and Network speeds and usage. This user data will only be stored on devices in an active call with the user and only be kept for the duration of the call (for IP and Network info) or less (temporarily in memory for Audio and Screen Capture). Users will be notified before joining a call that they will be sharing this information with all members of the call. This complies with GDPR's seven data protection principles. This app is not subject to PIPEDA as it is free, open-source, and has no underlying business model, meaning it does not constitute commercial activity.

---

8. Functional Requirements

P0 Requirements (Must have)
Users must be able to use the app for its basic intended purpose: being able to join calls, and either share their screen or view another person's screen. The app needs to work on Windows and allow users to set a fixed bitrate for their video stream.

P0 requirements include:

- Windows app with:
    - Ability to join calls with 1 other user
    - Ability to pick a display and share the user's entire screen
    - Ability to watch a shared screen in real time
    - Fixed bandwidth limits (download for receivers, upload for broadcaster)

P1 Requirements (High Priority)
These P1 features will greatly increase the usability and functionality of the app toward its target state, but are not prerequisites for the app to function. The two main focuses are the networking systems to allow multiple users to join calls together, and quality of life features/settings to make calls remain functional as more users join.

P1 requirements:

- Ability to join calls with up to 9 other users
- Symmetric multi-unicast audio and video broadcasting
    - i.e. everyone sends audio to each other, and everyone can view the broadcasters screen
    - stream will be sent out by simply duplicating stream to all users, instead of using a complex algorithm.
- When sharing, ability to select specific application or whole display to share
    - Ability to select broadcast framerate, resolution
- Dynamic bitrate scaling based on network speeds
- User settings and quality of life
    - Push-to-talk option
    - Audio input/output volume
    - Volume sliders to adjust volume for user in the call
    - broadcaster audio level
    - Framerate vs quality preference
        - i.e. should the stream video quality or framerate be decreased if there is not enough bandwidth
    - In call: Mute or deafen (mute mic and volume)
    - Display usernames

P2 Requirements (Medium Priority)

Our P2 requirements mainly consist of the network flow algorithm. This system will greatly increase performance, but also the complexity of our app. It is a novel feature that will help us solve some of the drawbacks of a peer-to-peer broadcasting system, and deliver exceptional performance in this domain.

P2 requirements:

- Network flow type algorithm
    - Complex algorithm to relay broadcast between non-streaming viewers to maximize stream quality
    - Allow relay setting, and maximum upload rate for non-streaming participants
    - Latency based restrictions settings
        - (i.e. only relay stream if it can be done under a certain latency)
- Linux support & ability for cross platform calls

P3 Requirements (Low Priority)
These requirements are nice to have options that will be investigated if time permits, but are not currently scheduled in the development plan.

P3 requirements:

- Noise suppression options
- Profile pictures for users
- MacOS support
- Ability for the viewer to interact with the shared screen
- GUI to show network mesh and data rates in real time
- Website to learn about and download software

---

9. Data Strategy & Quantitative Metrics

As this project does not use machine learning, testing focuses on running the application across different networks, devices and content types, with network emulation used to simulate poor connections.

The application will be tested on four network setups.

- **McMaster Wi-Fi** is a shared, congested campus network with a likely strict firewall, which makes it useful for testing performance under real-world congestion on a restrictive network.
- **Home Wi-Fi Networks** in different cities and on different ISPs will be used to test realistic internet calls with real latency, jitter and asymmetric bandwidth.
- **Gigabit Ethernet**, either direct or through a switch, provides a near-ideal baseline to determine application overhead.
- **Mixed Network Setups,** where peers in the same call are on different networks, will be used to test how the bandwidth allocation algorithm adapts to different network speeds.

The following validation metrics will be used to measure success. These metrics will only be collected during testing, except network speeds and video bitrates, which the application needs to adjust stream quality, none of this data will be collected or stored in the final application due to privacy concerns.

- **Video bitrate**: Each stream's bitrate is logged every second and compared against the number of viewers and the host's available upload bandwidth. The host's total upload must stay below 85% of its capacity and maintain at least 240p at 30 fps.

- **Network efficiency**: Efficiency is the share of transmitted data that is audio and video rather than overhead, measured with Wireshark. The target is at least 90% at 720p and 80% at 240p.
- **Resource usage**: CPU, GPU and RAM are recorded separately for hosts, viewers and relays. On the minimum spec, a viewer at 720p should use under 40% CPU and 500 MB of RAM, and a host sharing to four viewers should use under 60% CPU.
- **Video latency**: Latency is measured by showing a millisecond clock on the host's screen and recording it alongside the viewer's screen. The target is under 150 ms locally and 300 ms over the internet, with each relay hop adding no more than 50 ms.
- **Audio latency**: Latency is measured by recording a test tone at both the sender and receiver. One-way latency must stay under 150 ms, with audio and video in sync to within 80 ms.
- **Supporting metrics**: The app should keep at least 95% of its target frame rate and stay connected at 5% packet loss. The first video frame should appear within 5 seconds of joining, and at least 95% of connection attempts should succeed.

---

10. Non-Functional Requirements (NFRs)

The interface of our application will focus on the following factors:

1. **Efficiency:** The interface can't have any redundancies that would slow down or confuse the user trying to access the main features of the app.
   The following table represents the main functions of the app and the maximum number of actions that each takes to achieve, where an action is an interaction with an element on the screen or typing under 15 characters in a selected field:

Function	Maximum number of actions	Function priority
Create a room	1	P0
Share your screen	2	P0
Join a call	3	P0
Watch a screenshare	4	P0
Change your stream bitrate	4	P1
Change a default setting	3	P1

Change a ‘per user’ setting	5	P1

2. **Simplicity:** A simple interface is best suited for a goal of getting the user from point A (home screen) to point B (video sharing) without any unnecessary confusion. An important measurement behind simplicity is if a new user can access the basic functionality without needing any external help.
3. **Consistency:** The UI will follow a consistent layout and design to make the user experience as intuitive as possible. We will measure consistency through comparison of different pages of the app, where the buttons with similar functionality would have to be consistent across the pages in their color, shape and position.

The application will support up to 10 viewers per stream, maintaining at least 95% of the selected frame rate and resolution quality, given the viewers meet the recommended system and network requirements. The stream delay in these cases must be within 300ms.

The data and connections of each session will be encrypted with user authentication to avoid any ‘man in the middle’ attacks, aimed at reading data. Any repeated connection attempts will be limited to avoid DDOS attacks. The user information won’t be stored on a third party database, making it more secure.

In cases of packet loss of 5% or less, the app will keep running uninterrupted, adapting to the new conditions. The sessions will not be terminated if one of its users experiences any type of interruption. The app will receive periodic updates to account for driver and OS updates, maintaining its functionality.

---

11. Risk Management and Feasibility Analysis

The advanced features of this software including dynamic bandwidth based stream allocation and full desktop cross platform support may be infeasible given the time frame. Thus, we are using a risk matrix to document the feasibility risks to determine how to best allocate our time to minimize impact on the project.

Impact /	Minor	Moderate	Major

Likelihood			
Unlikely	Screen audio capture requires multiple additional libraries / modules. Per user audio limiting is infeasible.	User modifiable bandwidth limits are infeasible.	Sharing streams via multi-unicast to ~10 people is infeasible. Screen audio capture is infeasible. P2P based video calls are infeasible.
Moderate	A dynamic flow based relay algorithm for stream routing is infeasible.	Windows support is the only feasible option. Specific application / screen stream selection is infeasible.	Algorithmic bandwidth control for stream bitrates is infeasible.
Likely	Full cross platform support is too complex and infeasible. Allowing viewing users to interact with the sharers screen is infeasible.	(Intentionally Blank)	(Intentionally Blank)

Based on our analysis, there is nothing with a high likelihood of occurring that would have a major or moderate impact on the project's success. It is moderately likely that the algorithmic bandwidth control will be harder than expected, and this would have a major impact as it is one of its differentiating features. Along with this, certain features like screen audio capture, sharing streams to ~10 people, and P2P based video calls are unlikely to be infeasible, but would have a major impact on the project if they were. Along with this there is a moderate likelihood that Windows support, and no application selection for screen will be the only feasible options for the application with a moderate impact on the project. Finally, everything with low-impact and high likelihood for infeasibility will be set as a P3 requirement, and items in the other section are generally set as P2s. We will investigate the feasibility of various risks as our project develops and adjust the scope and development plan accordingly.

---

12. Team Communication & Meeting Cadence

Our team will hold semi-weekly meetings: online meetings on Mondays and in-person meetings on Wednesdays. The meeting records will be maintained under /meetings on

GitHub. The team will also schedule two supervisor check-ins and one TA check-in per semester.

Each meeting will have a duration of 1 hour, with fixed times, agreed upon based on each member's availability. The start times for the meetings are as follows: Monday - 8:00pm, Wednesday - 1:30pm. For any communication between the meetings, our team will use a Discord group chat.

Our meeting records must contain the following information: attendance, final decisions, new deadlines. If a person is absent from a meeting due to unforeseen circumstances, they will have to familiarize themselves with the record. The rest of the team will relay any unresolved questions or tasks that required the missing member's input.

---

13. Engineering Workflow & Version Control

Repository Structure:

The repository will contain the following main folders:

- /src with subfolders for each submodule in the application
- /tests for all tests, with subfolders for each code module
- /docs for all documentation and information about features
- /build for the working build of the project
- /tracking with a /meeting subfolder for meeting notes and attendance
    - Any other work tracking or decision justification notes will be put in this folder

Branching Strategy

We will use the GitFlow branching strategy, consisting of

- main branch for the latest release of the project
- a development branch that features will be created off of and merged into
- feature branches for work specific tasks
- release branches off of the development branch once sufficient work is done for testing and release

PRs will be required to merge feature branches into develop. Each PR must be reviewed by at least one other team member before being merged.
PRs will also be required to merge releases (or hotfixes) into main and will require a team member to double check and approve.

Continuous Integration/Continuous Deployment
GitHub actions will be used to automatically build the project and run all tests when a PR is created. If needed, a Docker container will be used to create an isolated environment for the app.

Issue Tracking
We will use GitHub issues to create and keep track of standard work items, as well as bugs and other tasks. A GitHub project with a Kanban board will be used to track assignment and progress on work.

Agile Framework
We will use an agile framework similar to Scrum. We will have semi-weekly meetings with all members for everyone to discuss their progress, ask questions, and get help if needed. We will also review our current trajectory as a team and make decisions about future steps. In the first weekly meeting, we will review the backlog and decide the priority of tasks to focus on next. In the second meeting, we will review progress, and team processes to determine if any improvements can be made. Instead of having fixed sprint deadlines to complete tasks and then review, we will continuously adjust priority and review a little bit each meeting.

---

14. Infrastructure and Compute Resources

Being peer-to-peer, SGVS requires low or zero overhead on server costs or cloud computing resources. It will be entirely open source, and all compute resources will fall on the user to provide themselves as a prerequisite of use. Amongst these requirements are:

- **Recent Windows operating system**
  Windows 10 or Windows 11. Other operating systems (MacOS, Linux) may be supported in the future, but at the start Windows will be required.
- **GPU**
  Found in most modern laptops and almost all desktop computers. Integrated graphics will work, but a dedicated GPU is preferred for best screen-sharing performance.

Otherwise, SGVS is non-applicable to the issue of locating and maintaining infrastructure and compute resources on the distributor's end. The backend will be written in Visual Studio with Windows API.

---

15. Technology Stack and Architectural Rationale

Core Architecture

The core of this application is based on a 3 module design:

- A processing module for collecting and processing video and audio inputs
- A networking module to handle sending data streams and managing bitrates
- An application module to handle the user interface and user interactions

These modules will function as three separate processes that interact through a series of shared memory buffers.

- A library will be developed in order to standardize the interaction between the modules
- The three modules act independently and don't know how to communication; The shared memory library will glue them together

This design was chosen as it just requires each segment of the application to follow a specification of inputs and outputs, but does not dictate how this must be implemented. This means that if any segment of the implementation of a module needs to be changed from the original design, it does not affect the other modules (as long as the way in which they communicate remains the same). Along with this, it divides the application into 3 distinct testable units that provide, making unit testing and debugging easier.

Processing Module

This module will handle the gathering and processing of video and audio inputs, and relay encoded video / audio frames to the other modules. This module will be built using C++ due to its ability to interact with the lower level APIs for low latency video and audio capture and processing. For Windows support, Windows.Graphics.Capture will be used to capture raw screen data and the WASAPI will be used to capture raw computer audio. The raw screen data will then be processed using FFmpeg through the Libav C libraries. This was chosen as FFmpeg is widely regarded as the standard for video encoding.

Networking Module

The networking module will be responsible for sending the audio and video data between users in the call. It will receive video and audio data from the processing module to transmit, and it will collect and send received data to the application module. It will manage all of the P2P connections and contain the algorithm that tracks the network conditions and chooses encoding settings to maximize bitrates, relaying this info to the processing module. The networking module will be built using Golang, and use WebRTC to handle the

real-time streaming connections. Golang was chosen as it is simple to use, performant, and optimized for concurrency. WebRTC was chosen as it is an industry standard framework that is open source and widely available.

Application Module
The application module takes data from the processing and networking modules and displays it to the user. It will be built using C# and the .NET framework due to it being performant, and the team's familiarity with C# and similar languages like Java. We will use Avalonia UI because of its seamless integration with C# and .NET and the smooth-looking UI it provides.

Shared Memory Library
The shared memory library will provide the underlying infrastructure to allow the three main modules to communicate. The core of this library will be implemented in C, with wrapped libraries for C++, Go, and C#. The main reason C is chosen as the core of the library is due to its ability for other languages (namely C++, Go, and C#) to import libraries written in C. This will allow the shared memory code to only be written once, reducing the likelihood for inconsistencies and errors. Another reason C is chosen is due to its simplicity when mapping to memory. It is very easy to understand what a segment of memory is storing and how it is being interpreted by a C program.

---

16. Proof of Concept Demonstration Plan

By the November 23rd Proof of Concept (PoC), we should have the following:

- Two demonstrators should be able to make a P2P connection and join an audio call
- Both demonstrators should be able to share their screen, and each should be able to watch the other demonstrator's screen
- Basic UI for joining and viewing screens
- Implemented only for Windows

---

17. Project Schedule & Gantt Chart

Oct 7th - Oct 18th: The team will complete setup and technical spikes, including the repository, CI pipeline and project structure.

Oct 19th - Nov 12th: The team will build the first MVP, which covers the P0 features. The individual modules (capture/encoding, networking and UI) will be built from Oct 19th to Nov

1st. They will then be connected into a working end-to-end application from Nov 2nd to Nov 12th.

Nov 12th - Nov 20th: The team will polish and test the MVP. This includes testing across McMaster Wi-Fi, home networks, and Ethernet, as well as fixing bugs as they are discovered.

Nov 21st - Nov 23rd: The team will prepare the demo script, rehearse and record a backup.

Nov 24th - Jan 25th: The team will work on system design documentation. This includes documenting the final architecture, module interfaces and network protocol, and planning the test approach for P1 and P2 features.

Nov 30th - Jan 10th: The team will set up a basic integration testing framework.

Jan 5th - Feb 14th: The team will build the second MVP, which covers the P1 features.

Feb 22nd - Mar 14th: The team will build the third MVP, which includes polishing the P1 features and implementing the P2 features.

Mar 15th - Mar 28th: The project will enter a feature freeze. This period is dedicated to testing, bug fixes, overall app polishing and user experience, as well as performance benchmarking against the Section 9 metrics.

Mar 29th - April 5th: The team will focus on Expo preparation, final documentation and rehearsal.

